

SORT: A Systematically Optimized Ranking Transformer for Industrial-scale Recommenders

Chunqi Wang*
Alibaba International
Digital Commercial Group
Hangzhou, China
shiyuan.wcq@alibaba-inc.com

Bingchao Wu
Alibaba International
Digital Commercial Group
Hangzhou, China
wubingchao.wbc@alibaba-inc.com

Taotian Pang
Alibaba International
Digital Commercial Group
Hangzhou, China
pangtaotian.ptt@alibaba-inc.com

Jiahao Wang
Alibaba International
Digital Commercial Group
Beijing, China
wjh177423@alibaba-inc.com

Jie Yang
Alibaba International
Digital Commercial Group
Beijing, China
jiehong.yj@alibaba-inc.com

Jia Liu
Alibaba International
Digital Commercial Group
Beijing, China
lj410672@alibaba-inc.com

Hao Zhang
Alibaba International
Digital Commercial Group
Beijing, China
zh138764@alibaba-inc.com

Hai Zhu
Alibaba International
Digital Commercial Group
Hangzhou, China
zhu hai@alibaba-inc.com

Lei Shen
Alibaba International
Digital Commercial Group
Hangzhou, China
kenny.sl@alibaba-inc.com

Shizhun Wang
Alibaba International
Digital Commercial Group
Beijing, China
shaoan.wsz@alibaba-inc.com

Bing Wang*
Alibaba International
Digital Commercial Group
Hangzhou, China
lingfeng.wb@alibaba-inc.com

Xiaoyi Zeng
Alibaba International
Digital Commercial Group
Hangzhou, China
yuanhan@alibaba-inc.com

Abstract

While Transformers have achieved remarkable success in LLMs through superior scalability, their application in industrial-scale ranking models remains nascent, hindered by the challenges of high feature sparsity and low label density. In this paper, we propose SORT (Systematically Optimized Ranking Transformer), a scalable model designed to bridge the gap between Transformers and industrial-scale ranking models. We address the high feature sparsity and low label density challenges through a series of optimizations, including request-centric sample organization, local attention, query pruning and generative pre-training. Furthermore, we introduce a suite of refinements to the tokenization, multi-head attention (MHA), and feed-forward network (FFN) modules, which collectively stabilize the training process and enlarge the model capacity. To maximize hardware efficiency, we optimize our training system to elevate the model FLOPs utilization (MFU) to 45%. Extensive experiments demonstrate that SORT outperforms strong baselines and exhibits excellent scalability across data size, model size and sequence length, while remaining flexible at integrating diverse features. Finally, online A/B testing in large-scale e-commerce scenarios confirms that SORT achieves significant gains in key business metrics—including orders (+7.47%), buyers (+6.67%) and GMV (+8.65%)—while simultaneously cutting latency by 62% and boosting

throughput nearly sevenfold (+589%). SORT has been fully deployed in production, serving all users on AliExpress.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Recommender Systems; Large Ranking Model; Transformer; Scaling Laws

ACM Reference Format:

Chunqi Wang, Bingchao Wu, Taotian Pang, Jiahao Wang, Jie Yang, Jia Liu, Hao Zhang, Hai Zhu, Lei Shen, Shizhun Wang, Bing Wang, and Xiaoyi Zeng. 2026. SORT: A Systematically Optimized Ranking Transformer for Industrial-scale Recommenders. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*, November 07–11, 2026, Rome, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3799682.3840109>

1 Introduction

The Transformer architecture[19] has established dominance in large language models (LLMs)[10, 18, 24], owing to its excellent scalability. However, its application to ranking models in industrial-scale recommenders remains in a nascent stage. For a long time, the landscape of ranking models has been dominated by specialized networks that focus on explicit feature interaction. For instance, DIN[30] is designed to extract patterns from sequential features, while DeepFM[6] and DCN[21] specialize in heterogeneous feature interaction. These networks are primarily designed to capture feature interaction by low-compute operations including vector-wise operations and low-dimensional matrix multiplication, which struggle to leverage the massive parallel computational capacity of

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CIKM '26, Rome, Italy*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2539-5/2026/11
<https://doi.org/10.1145/3799682.3840109>

modern accelerators and are also unable to capture high-dynamic data patterns. Given the critical role of ranking models in industrial recommenders and the huge success of LLMs, we propose rebuilding ranking models based on the Transformer architecture, paving the way for highly unified and scalable ranking models.

However, directly applying the Transformer architecture to ranking models is sub-optimal due to the inherent differences between recommendation and language modeling. A primary challenge lies in the synergy between **high feature sparsity** and **low label density**, which hinders the model’s ability to learn robust representations from discrete, massive feature spaces with infrequent supervisory signals. In language modeling, feature sparsity is relatively low due to the adoption of subword tokenization techniques such as BPE[14] and wordpiece[16], which constrain the vocabulary size to a manageable range (e.g., within hundreds of thousands). In the meanwhile, its label density is inherently high, as every position within the text sequence possesses a supervisory signal (i.e., next-token prediction) drawn from the entire vocabulary-sized search space. Conversely, ranking models must handle item vocabularies at the billion scale, necessitating enormous embedding tables and resulting in extremely high feature sparsity. The challenge is further defined by low label density, stemming from two key factors. First, the historical sequence makes up the majority of the input but lacks direct supervision, which is restricted solely to the target items. Second, these signals are binary and sparse by nature, where the vast majority of samples are negative (zero-valued), such as instances of non-clicks. This information asymmetry, where a massive parameter space is regularized by sparse binary labels, not only induces overfitting, but also leads to substantial computational waste on long historical sequences.

To address these issues, we introduce *request-centric sample organization*, *local attention*, and *query pruning* to mitigate the computational inefficiency caused by the long historical sequences. By reallocating the Transformer’s computation to label-relevant interactions, these techniques significantly reduce computational overhead. In addition, we incorporate *generative pre-training* to resolve the issues introduced by the information asymmetry of high feature sparsity and low label density. Since the sparse binary signals from target items are insufficient to regularize a billion-scale parameter space, this approach augments label density through generative training (i.e., next-item prediction), thereby effectively alleviating overfitting and stabilizing the training of Transformer-based ranking models. Complementing these optimizations, we further refine the Transformer architecture by enhancing the tokenization, multi-head attention layer (MHA) and feed-forward network (FFN) modules. For the tokenization module, we introduce *special tokens* as feature boundaries. Within the MHA layer, we introduce *QKNorm* and *attention gate* to stabilize the training process. For the FFN module, we replace the standard architecture with a mixture-of-experts (MoE) layer, thereby increasing model capacity without incurring additional computational costs.

We refer to the proposed approach as SORT, standing for **S**ystematically **O**ptimized **R**anking **T**ransformer. Extensive experiments show that SORT achieves significant performance gains compared with strong baselines. We also evaluate SORT’s scalability across multiple aspects: data size, model size and sequence

length, demonstrating that SORT exhibits excellent scalability. Finally, extensive online A/B testing across multiple scenarios on AliExpress demonstrates that SORT not only yields substantial business growth (+7.47% in orders, +6.67% in buyers, and +8.65% in GMV) but also drastically enhances serving efficiency, cutting latency by 62% and boosting throughput nearly sevenfold (+589%).

2 Related Work

Conventional deep learning recommendation models (DLRMs) for recommendation typically rely on specialized architectures to handle sequence features (e.g., DIN[30]) and extensive handcrafted heterogeneous features (e.g., DeepFM[6], DCM[21]). Unlike Transformers, these architectures lack scalability, resulting in limited model capacity. Consequently, patterns are primarily captured through the rote memorization of sparse embeddings (e.g., item embeddings). Previous studies have demonstrated that such massive embedding tables often impair generalization and lead to overfitting[20, 29].

Motivated by LLMs, HSTU[25] introduces scalable architectures into RMs. Specifically, HSTU treats ranking as a next-token prediction task and relies solely on a unified sequence as input. This modification amortizes the training cost since it allows processing multiple candidates within a single sample. Subsequent models, such as MTGR[7], GenRank[9] and this work, have also adopted a similar paradigm. Wukong[26] also introduces scalable architectures into ranking tasks. However, in contrast to HSTU, Wukong focuses on handling heterogeneous feature interaction. RankMixer[31] also falls into this category. Recently, OneTrans[28] introduced a Transformer variant that leverages independent parameters to model heterogeneous features and user sequence. The independent parameters can be regarded as rule-based routing MoE. Different from that, SORT avoids rule-based routing MoE instead using a data-driven routing one. To better position our work, one can envision a spectrum where traditional DLRMs—reliant on specialized feature engineering—occupy one end, and LLMs—characterized by unified, scalable architectures—occupy the other. Compared to prior efforts, SORT shifts significantly toward the LLM side of this spectrum. By embracing a more streamlined architecture, SORT departs from the complexity of domain-specific interaction modules, paving the way for a more generalized ranking paradigm.

Despite these architectural advancements, GPSD[20] identified that the presence of sparse item ID features continues to hinder the full realization of model scalability. To address this, GPSD proposed a framework that employs generative pre-training and sparse embedding freezing strategy. This framework effectively unlocks the scalability of Transformers on ranking tasks while simultaneously mitigating the one-epoch overfitting problem[29]. Therefore, we also incorporate GPSD in this work and prove its great effectiveness.

3 SORT

3.1 Preliminaries

3.1.1 Task. We formulate the ranking problem as a multi-objective binary classification task to simultaneously predict the probabilities of three primary user behaviors: click, cart, and purchase. We use AUC to measure the model performance.

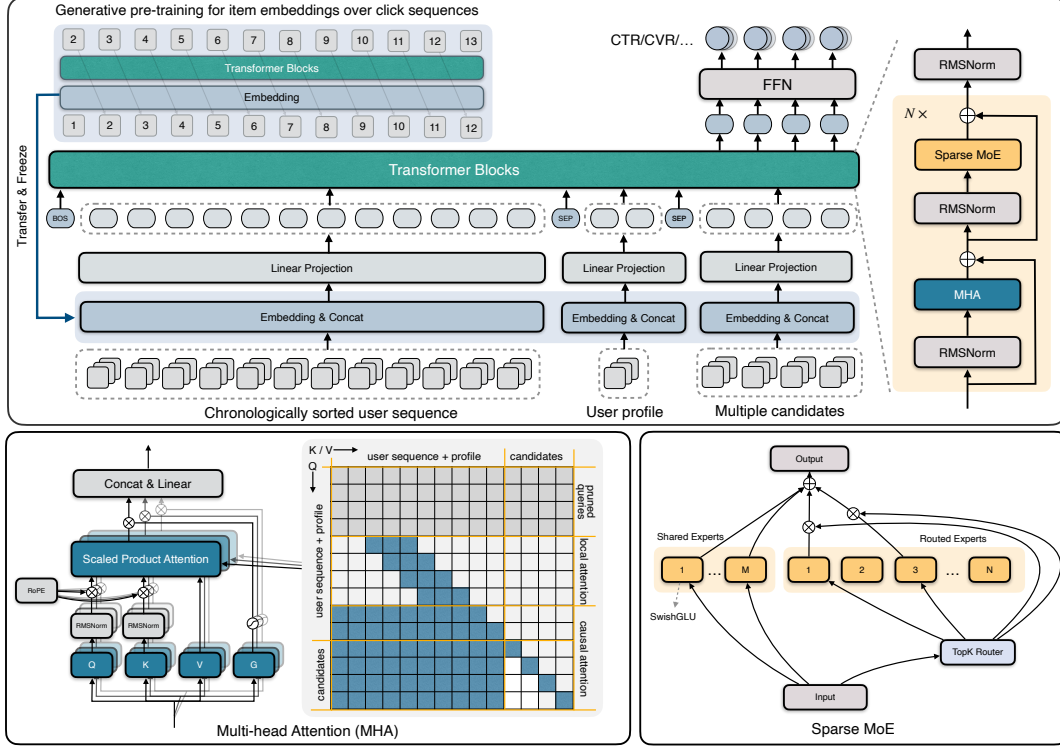


Figure 1: Overview of SORT. The input concatenates user history $\mathcal{H}$, profile $\mathcal{U}$, and candidates $\mathcal{C}$ with BOS/SEP tokens. Each block uses QKNorm and gated attention with a sparse mask, followed by a sparse MoE layer; query pruning progressively shortens the sequence across layers. The ranking head outputs multi-objective predictions on candidate tokens.

3.1.2 Data. Conventional ranking models typically operate on impression-centric samples (i.e., one candidate per sample). Under this setting, the model redundantly re-processes request-invariant features (e.g., user profiles and sequences) for each intra-request candidate, leading to unnecessary computational overhead. To optimize efficiency, we implement a request-centric data organization that encapsulates multiple intra-request candidates into a unified structure. Specifically, a request r involves a candidate set $C = \{c_j\}_{j=1}^N$ to be ranked, where each item c_j is characterized by item metadata, statistical features, and user-item cross features.

Simultaneously, the request r encompasses request-invariant features that are shared across all candidates in C . These features consist of two distinct components:

- User sequence ($\mathcal{H}$): A chronological sequence of historical behaviors $\mathcal{H} = (i_1, i_2, \dots, i_L)$, where each behavior i_t incorporates static item metadata and the associated interaction context (e.g., timestamp, action type, and scene).
- User profiles ($\mathcal{U}$): A combination of features integrating static user metadata and statistical features.

Instead of forming N impression-centric samples $\{\langle \mathcal{H}, \mathcal{U}, c_j \rangle\}_{j=1}^N$ for the request r , we define a unified request-centric sample $\mathcal{S}$ as follows:

$$\mathcal{S} = \langle \mathcal{H}, \mathcal{U}, C \rangle$$

3.2 Model

Our proposed model is built upon the Transformer architecture, systematically optimized for industrial ranking. Figure 1 provides an overview of the model.

Departing from the first Transformer[19], SORT directly integrates evolved best practices in LLMs[18]: Pre-Normalization[23] with RMSNorm [27], RoPE[17] for relative positioning, and SwishGLU[15] activations. Other key optimizations are detailed below.

3.2.1 Tokenization. In the tokenization phase, we generate a single token for each item (including both items in the user’s historical sequence and candidate items), while the user profile is mapped to multiple tokens. We also experimented with tokenizing candidate items into multiple tokens but a negligible performance difference was observed.

The tokenization pipeline consists of performing embedding lookup on multiple raw features, followed by concatenation, and finally applying linear projection and normalization to map the result to the target model dimension.

In addition to these feature-derived tokens, we introduce two special tokens: a special token, BOS, is prepended to the user history sequence, and two SEP tokens are inserted between distinct feature groups. These special tokens serve a dual purpose: they provide positional information to help the model identify sequence boundaries (i.e., the start and end), and simultaneously act as attention

sinks[22], absorbing excess attention scores. We observe that adding these special tokens significantly enhances model performance.

The final input sequence S is constructed by concatenating all aforementioned tokens in a specific order:

$$S = [\text{BOS}; \text{Tokenize}(\mathcal{H}); \text{SEP}; \text{Tokenize}(\mathcal{U}); \text{SEP}; \text{Tokenize}(\mathcal{C})]$$

3.2.2 Multi-head Attention. Multi-head Attention serves as the core operation of the Transformer, establishing direct dependencies between distinct tokens based on semantic similarity. We incorporate RoPE[17] to explicitly model relative positional information. To ensure candidates are processed independently, we apply a diagonal mask on the candidates and assign them an identical position ID, so that each candidate attends only to itself and the preceding history/profile tokens. To enhance training stability and model performance, we implement QKNorm[8], which applies an RMSNorm[27] layer to the queries and keys. Furthermore, drawing inspiration from HSTU[25] and gated attention[13], we introduce a gating operation following the scaled dot-product attention, a modification that yields further performance gains.

To mitigate the computational overhead associated with long user sequences, we utilize a sparse attention mask. Building upon the standard causal mask, we restrict the attention mechanism for the majority of tokens in the historical behavior sequence to local attention[1] while preserving standard causal attention for all candidate tokens and extra few tokens near the candidates. This design is highly conducive to processing very long user sequences, reducing the $O(L^2)$ time complexity to $O(W \cdot L)$, where L is the sequence length and W is the window size.

Finally, we leverage a query pruning strategy similar to OneTrans[28]. At each layer, we prune queries associated with tokens distant from the candidate tokens, resulting in progressively shorter sequences toward the top layers. At the final layer, we keep at most 128 non-candidate tokens. This modification almost halves the computational cost. Moreover, we see that this approach leads to unexpectedly better model performance compared to the non-pruning baseline, which can be attributed to the inductive bias of the inherent temporal decay in recommendation systems.

3.2.3 Feed-Forward Network. The Feed-Forward Network (FFN) constitutes another critical component of the Transformer architecture, designed to enhance the network’s non-linearity and capacity. Although the original Transformer[19] adopts ReLU activation function, later Transformers commonly use SwishGLU[15] for better performance, which is also adapted in our implementation. Another prevailing strategy for optimizing the FFN module is the sparse Mixture-of-Experts (MoE) framework. This approach deploys multiple experts and employs a routing network to assign routing weights, thereby activating only a specific subset of top- k experts for each input. The final output is obtained via the weighted aggregation of the selected experts’ outputs. The primary advantage of Sparse MoE lies in its ability to significantly scale up the number of parameters—thereby enhancing the model’s memorization capacity—without incurring additional computational overhead. This characteristic is particularly critical in recommendation scenarios, which demand strict adherence to low-latency constraints.

We conducted a comparative analysis between Switch-style MoE[4] and DeepSeek-style MoE[2]. The latter not only obviates the

need for tuning the auxiliary load balancing loss hyperparameter but also delivers slightly better performance as shown in Section 4.3.2. Consequently, we adopt DeepSeek-style MoE in our final architecture.

3.2.4 Ranking Head and Loss Function. To derive the final ranking scores, we apply an FFN to the output hiddens corresponding to the candidate item tokens. This FFN comprises a hidden layer with ReLU activation, followed by an output layer utilizing a sigmoid activation function to yield prediction for each objective. We employ a binary cross-entropy loss for each objective. The total training loss is computed as the weighted summation of the losses from all individual objectives.

3.2.5 Sparse Embedding Strategy. Feature sparsity presents a critical challenge in recommendation models, primarily stemming from the massive scale of users and items. Incorporating learnable embedding tables for these entities introduces an enormous number of parameters, which frequently leads to severe model overfitting[29].

To address this issue, we adopt distinct strategies for user and item representation. For users, we eschew explicit user id feature and instead characterize users through their historical behavior sequences and profile (e.g., age, location). For items, we adopt a generative pre-training paradigm proposed in GPSD[20] to mitigate sparse-embedding overfitting by augmenting label density via next-item supervision. We investigate two strategies: *sparse transfer* and *sparse transfer + sparse freeze*. The first involves initializing with pre-trained sparse embeddings and fine-tuning them during the training of the ranking model; the second additionally freezes the sparse embeddings throughout the training process.

3.3 Infrastructure

Our training system is built upon RecIS [32], with the MoE module implemented via Megablocks[5]. The architecture comprises a sparse module bottlenecked by I/O and memory access, and a dense module bottlenecked by computation. For the sparse module, beyond conventional optimizations (operator fusion, merged memory accesses, dynamic embedding), we introduce multi-process-group communication (MPGC), which schedules feature-embedding traffic across parallel process groups to hide communication latency. For the dense module, since existing attention operators including SDPA[12] and FlashAttention[3] are ill-suited to our structurally complex sparse masks, we develop a general sparse attention operator that uses block-wise computation with mask preloading to skip fully masked blocks, complemented by mixed-precision computing and gradient accumulation. Together, these optimizations elevate MFU from 13% to 22% for models at the base scale. Notably, for the large model, MFU further reaches 45%, demonstrating that our training system scales efficiently with model size.

For inference, we convert dynamic graphs to static graphs via `torch.export` and AOTInductor[11], and apply three classes of enhancements: (i) high-performance sparse attention kernels tailored to diverse masks, (ii) operator fusion that consolidates self-attention linear layers and merges fragmented trivial operators, and (iii) general LLM-style optimizations including half-precision computing, KV caching, and multi-context/multi-stream execution. Together,

	#Requests	#Users	#Impressions
Train	0.6B	50M	9B
Test	20M	4M	0.3B

Table 1: Dataset statistics.

Scale	Model	Depth	Width	Intermediate Size
Base	Transformer	6	512	1280
	OneTrans	6	512	1280
	HSTU	6	768	-
	SORT	6	512	1280
Small	Transformer	4	256	640
	SORT	4	256	640
Large	Transformer	12	1024	2560
	SORT	12	1024	2560

Table 2: Model settings. Because SORT is built on MoE, the effective intermediate size is the product of the number of the activated experts for each token and the intermediate size of each expert. HSTU lacks an FFN block, so its intermediate size is omitted in the table. To match the parameter number and FLOPs of other models, we accordingly widen HSTU.

these deliver a 29.4% throughput increase and a 29.3% latency reduction over the unoptimized baseline.

4 Experiments

4.1 Settings

4.1.1 Dataset. We collect the training set from the user logs of an industrial recommender system over the past months and reserve the subsequent day for evaluation. The dataset statistics are listed in Table 1.

4.1.2 Baseline. We select several strong baseline models, including Transformer, HSTU[25] and OneTrans[28]. The baseline Transformer is a standard implementation that lacks the specialized features of SORT, including local attention, query pruning, MoE, attention gate, and QKNorm. We carefully set the size of each model for fair comparisons, as listed in Table 2.

4.1.3 Training Settings. We use AdamW optimizer to train all the models. We set $(\beta_1, \beta_2) = (0.9, 0.99)$ and weight decay to 0.01. We use learning rates of 5e-4, 2e-4 and 1e-4 for small, base and large models respectively. We use a batch size of 12K in all experiments. Unless otherwise specified, we train each model by one epoch with a max user historical sequence length of 1K. Except for retaining FP32 precision for the ranking head and MoE routers, we use automatic BF16 mixed precision to train the models. Each model is trained on 4 to 32 A100 GPUs for 1 to 10 days, depending on the model size.

4.2 Effectiveness of Generative Pre-training

We verified the effectiveness of the generative pre-training paradigm on the baseline Transformer architecture. The results are presented in Table 3. It can be observed that the strategy of simply transferring pre-trained embeddings (*sparse transfer*) yields no performance improvements but even led to a slight degradation. In

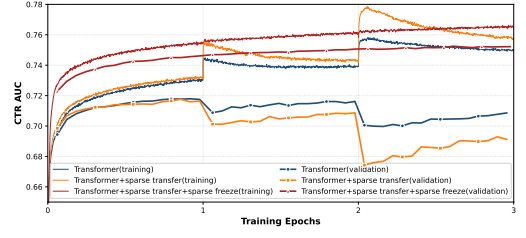


Figure 2: Training and validation AUC curves across multiple epochs under different strategies.

Model	CTR		CVR		AddCart	
	AUC	Imp	AUC	Imp	AUC	Imp
Transformer (from scratch)	0.7175	-	0.8959	-	0.8455	-
Transformer +sparse transfer	0.7162	-0.13pt	0.8943	-0.16pt	0.8449	-0.06pt
Transformer +sparse transfer +sparse freeze	0.7456	+2.81pt	0.9093	+1.34pt	0.8663	+2.08pt

Table 3: Impact of generative pre-training. Imp stands for absolute AUC improvements.

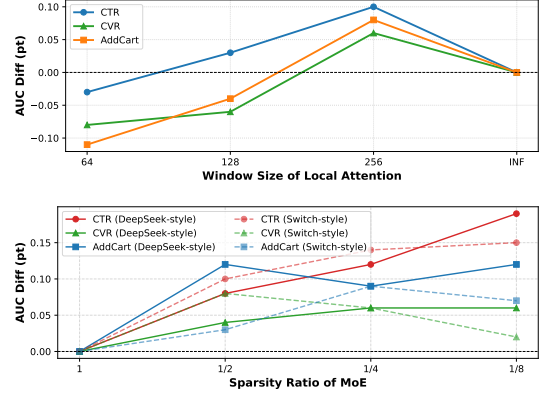


Figure 3: Impacts of various hyperparameter options.

contrast, additionally freezing the pre-trained embeddings (*sparse freeze*) yields a significant performance boost. Another advantage of *sparse freeze* is that it allows multi-epoch training without severe overfitting. As we can see for the CTR task in Figure 2, the training and validation AUC curves both keep growing smoothly across multiple epochs. Given the effectiveness, we adopt *sparse transfer + sparse freeze* as the default strategy for the remainder of this paper.

4.3 Hyperparameter Exploration

In this section, we explore two important hyperparameters, i.e. window size of local attention and sparsity ratio of MoE, trying to figure out their optimal values. The results are presented in Figure 3.

4.3.1 Local Attention Window Size. This hyperparameter controls how many tokens a token can at most attend to. If the value is small, it can only attend to very few tokens nearby thus lacks long-term dependencies. Conversely, if the value is large, it can attend to too

many tokens thus cost much computation. We select three values for this hyperparameter: 64, 128, and 256. It is evident that 256 achieves the superior results across all tested values. Surprisingly, its performance even surpasses that of the vanilla causal attention, which suggests that an appropriately constrained local window acts as a beneficial inductive bias, filtering out distant noise and allowing the model to focus on the most semantically relevant local context.

4.3.2 MoE Sparsity Ratio. This hyperparameter is a ratio calculated by

$$\frac{\text{The number of activated experts per token}}{\text{Total number of experts}},$$

which controls how sparse the MoE is. A dense model without MoE can be regarded as a special case where the sparsity ratio equals 1. We select three values for this hyperparameter: 1/2, 1/4, and 1/8. Besides, we also compare two popular MoE implementations: Switch-style MoE and DeepSeek-style MoE. In summary, the DeepSeek-style MoE demonstrates more effectiveness compared to the Switch-style counterpart. Notably, it simplifies the training process by obviating the effort required for hyperparameter tuning. The optimal result is observed at a sparsity ratio of 1/8.

4.4 Architectural Ablation and Comparison

We validate the efficacy of individual optimizations integrated into the base Transformer. As illustrated in Table 4, key techniques including *special tokens*, *query pruning*, and *MoE*, demonstrate substantial independent gains, with *special tokens* and *query pruning* notably boosting CTR-AUC by +0.33pt and +0.26pt respectively in the base scale. To visualize the impact of *special tokens*, we show the attention heatmap in Figure 4, which demonstrates that the BOS token absorbs considerable attention weights, acting as attention sink[22]. We also observe that *QKNorm* effectively mitigates the fluctuations of the attention weights, which is beneficial for the stability of the model training. Other optimizations, such as *attention gate* and *local attention*, also demonstrate substantial gains. The synergy of these optimizations culminates in our SORT model, which achieves state-of-the-art results across all primary tasks, significantly outperforming both standard Transformer and specialized architectures like HSTU and OneTrans. Notably, while *local attention* only reduces FLOPs from 43G to 40G at a sequence length of 1K, it becomes crucial for longer sequences.

Beyond static metrics, the robust scalability of SORT is a highlight of our empirical findings. Across small, base, and large scales, SORT consistently maintains a significant advantage over the Transformer baseline. Notably, at the large scale, SORT achieves +0.51pt improvement in CTR-AUC, while requiring substantially fewer FLOPs (188G vs. 322G). As illustrated in Figure 5, across all scales, the training AUC curves for SORT consistently stay above their Transformer counterparts, demonstrating both its remarkable performance and sample efficiency.

4.5 Data, Model and Sequence Length Scaling

We conducted scaling experiments across three dimensions: data size, model size (number of parameters), and sequence length. Unlike LLMs, which benefit from a virtually inexhaustible supply of

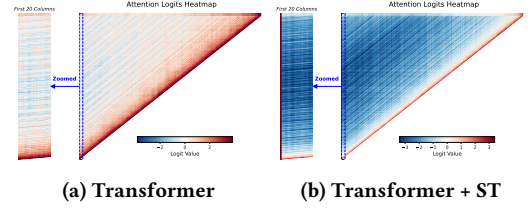


Figure 4: Heatmap visualization of the impact of special token on averaged attention logits at layer 0.

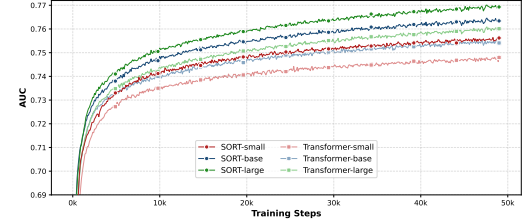


Figure 5: Training AUC curves for CTR task.

data, recommender systems are inherently constrained by the volume of user interactions. Consequently, the available data within a given timeframe is finite. To address this limitation, we explore the following two strategies to expand the effective data size. The first method is to train the model for multiple epochs on the same dataset. Benefiting from generative pre-training and frozen embeddings, the model avoids the common issue of overfitting during multiple epochs, as we see in Figure 2. The second approach involves combining data from other scenarios with the target set to improve model performance on the target scenario. In this setup, the specific scenario identifier is encoded as an input feature, enabling the model to distinguish between different data sources. In summary, we investigate the scaling effects under the following experimental configurations:

- Data scaling (multi-epoch): Training for 1, 2, and 3 epochs.
- Data scaling (multi-scenario): Comparison between single-scenario and multi-scenario settings.
- Model scaling: Model variants of small, base, and large sizes.
- Sequence length scaling: Sequence lengths of 256, 512, 1K, 2K and 4K.

The experimental results are illustrated in Figure 6. We observe that increasing the data size, model size, or sequence length consistently leads to performance gains, thereby validating the strong scalability of our proposed method. Overall, data scaling proves to be the most effective, exhibiting a steeper upward slope in performance improvement. This finding suggests that in production, the best practice is to assign higher priority to data scaling, followed by the sequence length scaling and model scaling, to achieve better performance under a given computational budget.

4.6 Online A/B Testing

To validate the effectiveness of SORT, we conduct extensive online A/B testing in four main recommendation scenarios on AliExpress, including HomePage, DetailPage, ShoppingCart and AfterPayment.

Scale	Model	CTR		CVR		AddCart		Params	FLOPs
		AUC	Imp	AUC	Imp	AUC	Imp		
Base	Transformer	0.7456	-	0.9093	-	0.8663	-	18M	43G
	HSTU	0.7438	-0.18pt	0.9070	-0.23pt	0.8644	-0.19pt	18M	43G
	OneTrans	0.7476	+0.20pt	0.9103	+0.10pt	0.8677	+0.14pt	54M	24G
	Transformer+special token (ST)	0.7489	+0.33pt	0.9113	+0.20pt	0.8685	+0.22pt	18M	43G
	Transformer+local attention (LA)	0.7466	+0.10pt	0.9099	+0.06pt	0.8671	+0.08pt	18M	40G
	Transformer+query pruning (QP)	0.7482	+0.26pt	0.9112	+0.19pt	0.8676	+0.13pt	18M	24G
	Transformer+attention gate (AG)	0.7477	+0.21pt	0.9097	+0.04pt	0.8672	+0.09pt	20M	47G
	Transformer+QKNorm (QKN)	0.7468	+0.12pt	0.9104	+0.11pt	0.8670	+0.07pt	18M	43G
	Transformer+MoE	0.7475	+0.19pt	0.9099	+0.06pt	0.8675	+0.12pt	83M	43G
	SORT(Transformer+ST+LA+QP+AG+QKN+MoE)	0.7497	+0.41pt	0.9118	+0.25pt	0.8694	+0.31pt	85M	24G
Small	Transformer	0.7424	-	0.9049	-	0.8612	-	3M	8G
	SORT	0.7465	+0.41pt	0.9082	+0.33pt	0.8656	+0.44pt	14M	4G
Large	Transformer	0.7485	-	0.9122	-	0.8695	-	144M	322G
	SORT	0.7536	+0.51pt	0.9156	+0.34pt	0.8736	+0.41pt	685M	188G

Table 4: Comparison of different models. Results are grouped by model scale for fair comparison. Bold indicates the best performance within each group. Imp stands for absolute improvements over the Transformer baseline within each scale. Note that FLOPs are calculated for a single sample forward pass, and Params refers to the number of parameters excluding embeddings. For all models, the number of embedding parameters is about 20B.

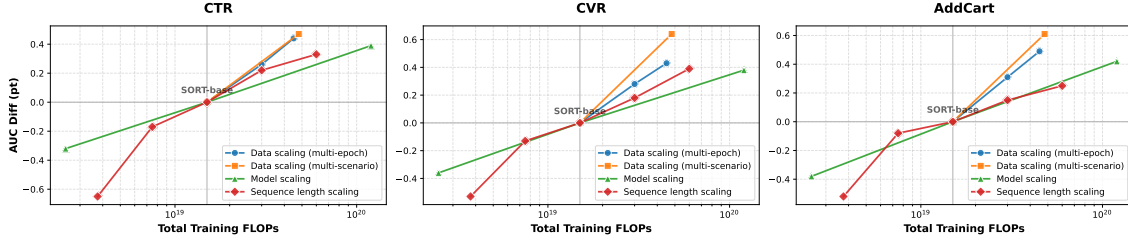


Figure 6: Scaling SORT over data size, model size and sequence length. The SORT-base model is trained on single-scenario data for one epoch, with sequence length of 1K.

Table 5: Online A/B test results over a one-month duration. $\uparrow$ ($\downarrow$) indicates higher (lower) values are preferred.

Scenario	Orders $\uparrow$	Buyers $\uparrow$	GMV $\uparrow$	Latency $\downarrow$	Throughput $\uparrow$
HomePage	+2.37%	+2.58%	+10.01%	-62%	+325%
DetailPage	+8.99%	+7.46%	+11.49%	-51%	+405%
ShoppingCart	+7.10%	+6.74%	+6.98%	-64%	+448%
AfterPayment	+11.43%	+9.91%	+6.12%	-72%	+1180%
Macro Average	+7.47%	+6.67%	+8.65%	-62%	+589%

The production baseline utilizes the conventional DLRM architecture, which has been incrementally trained on data spanning over a year. In contrast, the deployed SORT model, which employs a base-scale configuration, is trained using only recent three months' data.

The experimental results (shown in Table 5) demonstrate that SORT achieves significant improvements in both effectiveness and efficiency. In terms of effectiveness, SORT yields substantial gains across multiple key performance indicators across all scenarios. On average, SORT achieves remarkable improvements of 7.47%, 6.67%, and 8.65% in orders, buyers, and GMV, respectively. This highlights the ability of SORT to accurately model user interests and improve the core metrics of the recommender system. Regarding serving efficiency, SORT achieves an average 62% reduction in

latency and an average 589% boost in throughput in four scenarios. The improved efficiency is attributed to a synergy of data, model, and system-level optimizations. Specifically, the request-centric data paradigm enables the reuse of shared user profile and historical sequence across all candidates in a single pass. Furthermore, the unified Transformer architecture mitigates the computational overhead associated with fragmented operators. These factors, combined with our optimized inference system, collectively result in significant gains in efficiency.

5 Conclusion

We present SORT, a systematically optimized ranking Transformer that bridges the gap between Transformers and industrial-scale ranking. Targeting high feature sparsity and low label density, SORT combines request-centric sample organization, local attention with query pruning, generative pre-training, and architectural refinements (special tokens, QKNorm, attention gate, sparse MoE) into a unified, hardware-efficient design. SORT outperforms strong baselines and scales smoothly along data, model size and sequence length, with data scaling yielding the steepest gains under a fixed compute budget. Fully deployed on AliExpress, SORT delivers substantial business gains (+8.65% GMV) while cutting latency by 62% and boosting throughput nearly sevenfold.

6 GenAI Usage Disclosure

Generative AI tools (Qwen, Gemini) were used solely for grammatical editing/polishing of the text. All content, ideas, and data analysis remain the original work of the authors.

References

- [1] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [2] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [3] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [5] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems* 5 (2023), 288–304.
- [6] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).
- [7] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, et al. 2025. Mtgr: Industrial-scale generative recommendation framework in meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5731–5738.
- [8] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. 2020. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 4246–4253.
- [9] Yanhua Huang, Yuqi Chen, Xiong Cao, Rui Yang, Mingliang Qi, Yinghao Zhu, Qingchang Han, Yaowei Liu, Zhaoyu Liu, Xuefeng Yao, et al. 2025. Towards Large-scale Generative Ranking. *arXiv preprint arXiv:2505.04180* (2025).
- [10] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [11] PyTorch Authors. 2024. AOTInductor. https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch_compiler_aot_inductor.html. Accessed: 2025-02-10.
- [12] PyTorch Authors. 2024. Scaled Dot-Product Attention (SDPA). https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html. Accessed: 2025-02-10.
- [13] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, et al. 2025. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. *arXiv preprint arXiv:2505.06708* (2025).
- [14] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*. 1715–1725.
- [15] Noam Shazeer. 2020. Glue variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).
- [16] Xinying Song, Alex Salcianu, Yang Song, Dave Dopson, and Denny Zhou. 2021. Fast wordpiece tokenization. In *Proceedings of the 2021 conference on empirical methods in natural language processing*. 2089–2103.
- [17] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [18] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [20] Chunqi Wang, Bingchao Wu, Zheng Chen, Lei Shen, Bing Wang, and Xiaoyi Zeng. 2025. Scaling transformers for discriminative recommendation via generative pretraining. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2893–2903.
- [21] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.
- [22] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
- [23] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International conference on machine learning*. PMLR, 10524–10533.
- [24] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [25] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [26] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. 2024. Wukong: Towards a scaling law for large-scale recommendation. *arXiv preprint arXiv:2403.02545* (2024).
- [27] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).
- [28] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixiu Sun. 2025. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. *arXiv preprint arXiv:2510.26104* (2025).
- [29] Zhao-Yu Zhang, Xiang-Rong Sheng, Yujing Zhang, Biye Jiang, Shuguang Han, Hongbo Deng, and Bo Zheng. 2022. Towards understanding the overfitting phenomenon of deep click-through rate models. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 2671–2680.
- [30] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
- [31] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.
- [32] Hua Zong, Qingtao Zeng, Zhengxiong Zhou, Zhihua Han, Zhensong Yan, Mingjie Liu, Hechen Sun, Jiawei Liu, Yiwen Hu, Qi Wang, et al. 2025. ReclS: Sparse to Dense, A Unified Training Framework for Recommendation Models. *arXiv preprint arXiv:2509.20883* (2025).